Prüfen und Migrieren

Lernunterlage: Ein Befehl prüft alles, und die Datenbank wandert auf Identity um

1 Worum es geht

Mehrere Änderungen (Admin-Schutz, Frontend-Login, API-Schlüssel, Tuya-Abgleich, PWA, Auslieferung des Frontends, Umstellung auf ASP.NET Core Identity) wurden ohne .NET SDK geschrieben und waren **nie gebaut und nie getestet**. Bevor man darauf aufbaut, muss der Stand einmal vollständig geprüft werden. Dazu gibt es jetzt **ein** Skript: `./pruefen.sh`.

> **Merksatz:** Ein Schritt gilt erst als erledigt, wenn Build und Tests wirklich gelaufen sind. Geschriebener, aber nie kompilierter Code ist nur eine Vermutung.

2 Das Prüfskript pruefen.sh

Schritt	Befehl	Was er prüft
Werkzeuge	dotnet tool restore	Holt dotnet-ef in der Version aus dotnet-tools.json
Backend bauen	dotnet build	Kompiliert API, Tests und BlinkSpike: Tippfehler, falsche Typen, fehlende using
Migrationen	dotnet ef migrations has-pending-model-changes	Hat sich das Datenmodell geändert, ohne dass es eine Migration gibt?
Tests	dotnet test --no-build	Alle xUnit-Tests, ohne ein zweites Mal zu bauen
Pakete	npm ci	Installiert genau die Versionen aus package-lock.json
Frontend bauen	npm run build	tsc -b prüft die Typen, vite build baut die fertigen Dateien
Lint	npm run lint	oxlint sucht typische Fehler und schlechten Stil

So ist es gebaut

```
set -uo pipefail                      # -u: unbekannte Variable = Fehler
                                      # pipefail: Fehler in einer Pipe zählt
schritt() {
    local name="$1"; shift
    "$@" 2>&1 | tee -a "$LOG"         # anzeigen UND ins Log schreiben
    local status=${PIPESTATUS[0]}     # Exit-Code des Befehls, nicht von tee
    if [ "$status" -eq 0 ]; then ergebnisse+=("OK      $name")
    else ergebnisse+=("FEHLER  $name"); fehler=1; fi
}
schritt "dotnet build" dotnet build --nologo
...
exit "$fehler"                        # 0 = alles grün, 1 = mindestens ein Fehler
```

- **Ohne `set -e`:** Das Skript soll nach einem Fehler weiterlaufen, damit man alle Probleme auf einmal sieht und nicht eins nach dem anderen.
- **Exit-Code:** 0 heißt Erfolg, alles andere Fehler. So kann später auch ein anderes Programm (z. B. eine CI-Pipeline auf GitHub) das Skript aufrufen und auswerten.

- **PIPESTATUS:** Bei `befehl | tee` wäre `$?` der Exit-Code von tee (fast immer 0). `${PIPESTATUS[0]}` ist der des ersten Befehls.
- **pruefen.log:** Die ganze Ausgabe landet in einer Datei, die in .gitignore steht. Man kann sie weitergeben, ohne Text aus dem Terminal zu kopieren.

Ergebnis am 31.10.2026

Schritt	Ergebnis
dotnet build	OK, 0 Warnungen, 0 Fehler
Migrationen vollständig	OK (nach dem Anlegen von IdentityUmgestellt)
dotnet test	OK, 103 von 103 Tests bestanden
npm run build	OK (264 kB JavaScript, 80 kB gezippt)
npm run lint	OK, 0 Warnungen, 0 Fehler

3 Migrationen: Datenbank und Code im Gleichschritt

EF Core beschreibt das Datenmodell im Code (Klassen, `OnModelCreating`). Eine **Migration** ist die Anleitung, wie die echte Datenbank von einem Stand zum nächsten kommt. Home Control wendet fehlende Migrationen beim Start automatisch an (`db.Database.Migrate()` in Program.cs).

Datei	Inhalt
2026..._Name.cs	Up() und Down(): was beim Anwenden bzw. Zurücknehmen passiert
2026..._Name.Designer.cs	Das vollständige Modell zu diesem Zeitpunkt (von EF erzeugt, nicht anfassen)
ApplicationDbContextModelSnapshot.cs	Der aktuelle Stand. Daran erkennt EF, was sich seit der letzten Migration geändert hat

Regel: Alte Migrationen werden nie verändert. Jede Änderung am Modell bekommt eine neue Migration mit sprechendem Namen.

Was IdentityUmgestellt macht

Aus der eigenen Klasse `User` wurde `User : IdentityUser<Guid>`, der DbContext erbt von `IdentityDbContext`. Deshalb erzeugt EF:

- Tabelle **Users** wird in **AspNetUsers** umbenannt, die Spalte **Role** fällt weg.
- Neue Spalten von Identity: NormalizedEmail, NormalizedUserName, SecurityStamp, ConcurrencyStamp, AccessFailedCount, LockoutEnabled, LockoutEnd, EmailConfirmed, PhoneNumber, TwoFactorEnabled.
- Neue Tabellen: AspNetRoles, AspNetUserRoles (wer hat welche Rolle), AspNetUserClaims, AspNetUserLogins, AspNetUserTokens, AspNetRoleClaims.
- Der Fremdschlüssel der RefreshTokens zeigt jetzt auf AspNetUsers.

„An operation was scaffolded that may result in the loss of data“: EF warnt, weil die Spalte Role gelöscht wird. Hier ist das gewollt. Die Rollen stehen jetzt in AspNetUserRoles, und die alten Benutzer hat `identity-vorbereiten.sh` vorher bewusst entfernt. Bei so einer Warnung immer die Migration lesen, bevor man sie anwendet.

SQLite und Tabellenumbau

SQLite kann viele Änderungen nicht direkt (Primärschlüssel ändern, Spaltentyp ändern). EF Core baut die Tabelle dann automatisch neu auf: neue Tabelle anlegen, Daten kopieren, alte löschen, umbenennen. Das passiert unsichtbar beim Anwenden der Migration.

4 Die Reihenfolge, die funktioniert hat

```
git fetch origin
git switch feature/fertigstellen        # aktueller main + Prüfskript
./identity-vorbereiten.sh               # Datenbank sichern, alte Benutzer löschen
dotnet tool restore
dotnet ef migrations add IdentityUmgestellt --project backend/HomeControl.Api
./pruefen.sh                            # alles prüfen
git add backend/HomeControl.Api/Migrations
git commit -m "Migration IdentityUmgestellt ergänzt"
git push
./app-starten.sh                        # wendet die Migration auf die echte DB an
```

- **Warum lokal?** `migrations add` muss das Projekt bauen und braucht das .NET SDK. Die Migration ist danach normaler Code und gehört ins Repo.
- **Warum zuerst sichern?** Eine Migration verändert die echte Datenbank. Die Sicherung liegt als `homecontrol.db.vor-identity.bak` neben der Datenbank.
- **Warum ein eigener Branch?** main bleibt lauffähig. Erst wenn alles grün ist, kommt der Pull Request.

5 Kurzantworten für Rückfragen

Frage	Antwort
Woher weißt du, dass der Stand funktioniert?	./pruefen.sh: Build, 103 Tests, Frontend-Build und Lint laufen in einem Durchgang grün.
Was ist eine Migration?	Eine versionierte Anleitung, wie die Datenbank zum aktuellen Datenmodell passt. Sie liegt als C#-Code im Repo.
Was passiert, wenn eine Migration fehlt?	has-pending-model-changes schlägt fehl, und Migrate() würde beim Start eine Ausnahme werfen.
Warum testen die Endpunkt-Tests auch die Migrationen?	WebApplicationFactory startet die echte App auf einer frischen SQLite-Datei. Program.cs wendet dabei alle Migrationen an.
Warum npm ci statt npm install?	npm ci hält sich exakt an package-lock.json und ändert sie nie. So baut jeder Rechner dieselben Versionen.

6 Merksätze

- Erst prüfen, dann weiterbauen. Ein Befehl für alles: ./pruefen.sh.
- Exit-Code 0 = Erfolg. Bei Pipes zählt ${PIPESTATUS[0]}.
- Modell geändert → neue Migration. Alte Migrationen nie anfassen.
- Warnung „loss of data“ → Migration lesen, Datenbank vorher sichern.
- Migrationen gehören ins Repo, die Datenbank (homecontrol.db) nicht.